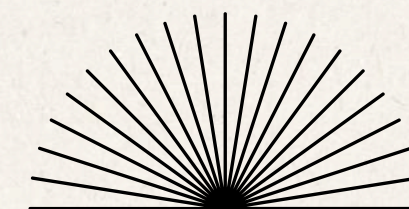


TRAVAUX D'ÉTUDES ET DE RECHERCHE

Jeu Pacman - Soutenance Finale

PRÉSENTÉ PAR :

Daboussi Akram & Bensallah Younes



Sommaire

03	Algorithme de Génération de Labyrinthe
14	Infrastructure & DevOps : Vue d'ensemble
15	Modélisation du Jeu : L'essentiel
16	IA Fantômes : Algorithme Glouton
29	IA Fantômes : Algorithme A*
30	Gestion du Jeu
33	IA Pac-Man : Minimax.. ?
37	IA Pac-Man : Minimax par copie d'états
41	Tests de Performance

Contraintes et Spécification du Labyrinthe Pac-Man

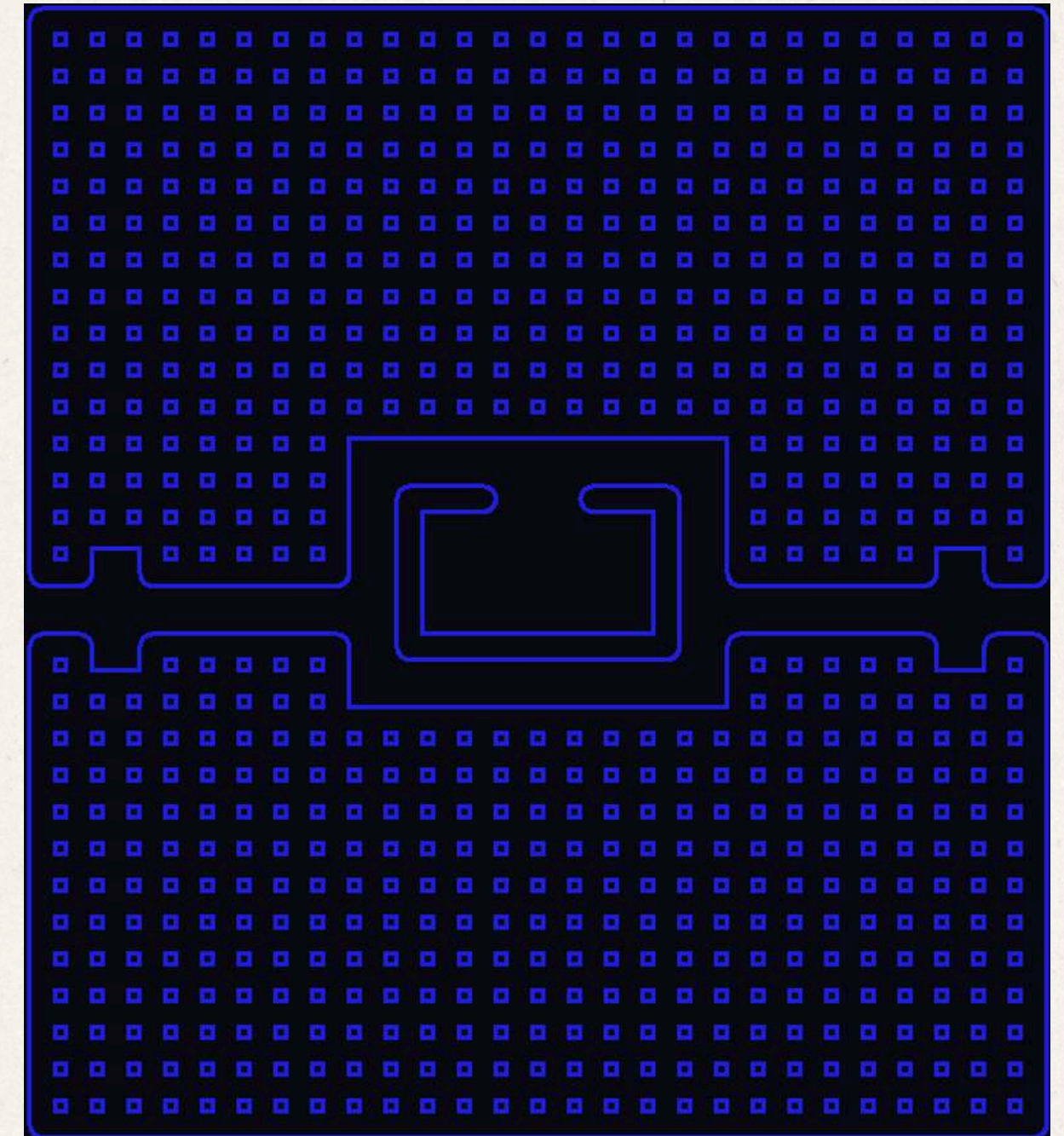
- Symétrie horizontale
- Gabarit fixe
- Graphe connexe
- Absence de culs-de-sac

Etape 1 : Placer les structures fixes

- ***Etats possibles*** = {MUR, SOL, MUR_PERMANENT, GHOT_HOUSE, TUNNEL, PORTE}
- Appel de applyTemplate() pour figer les structures fixes avant la génération
- Remplissage du reste de la grille avec des MUR prêts à être creusés.

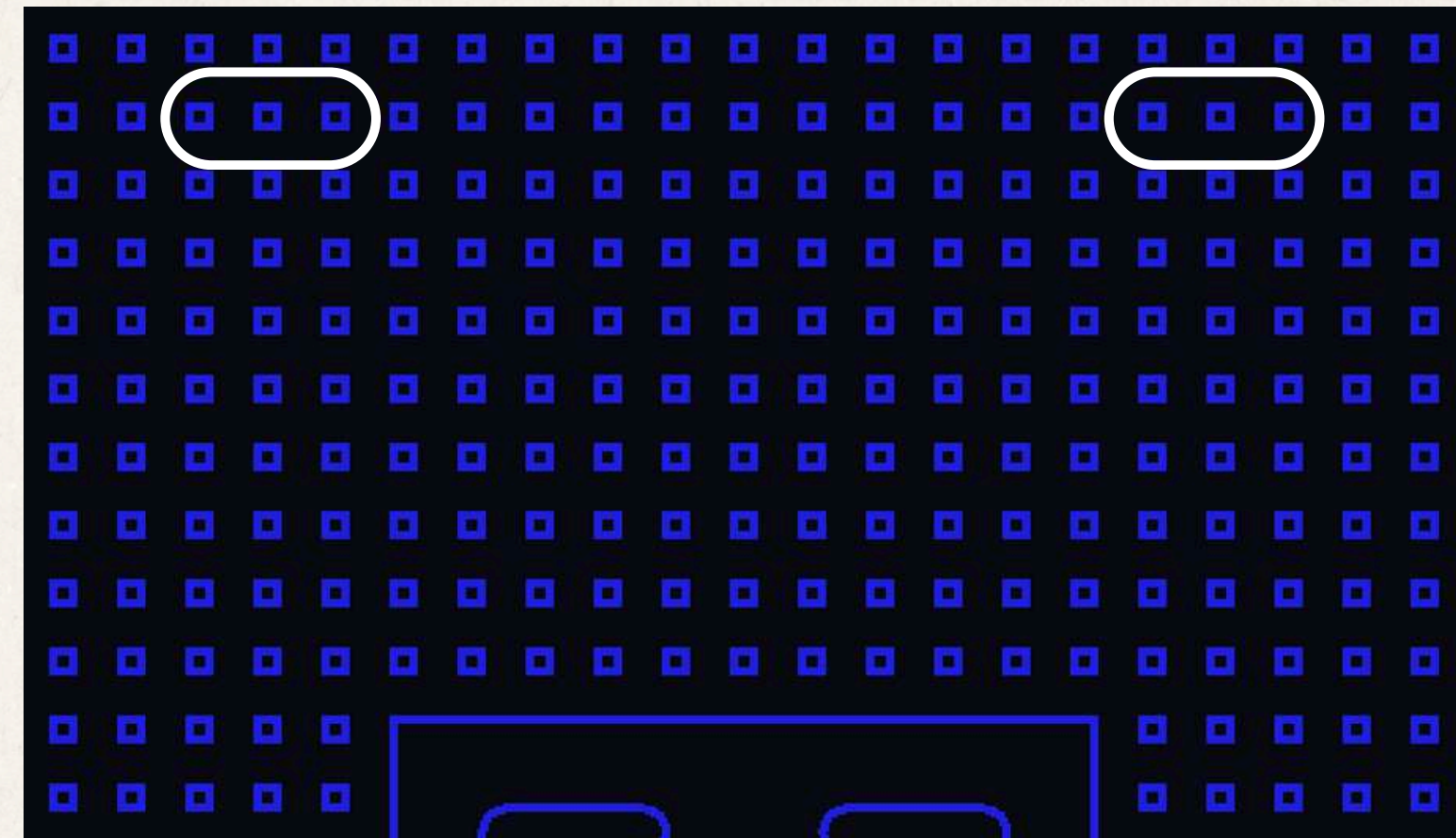
Etape 1 : Placer les structures fixes

- ***Etats possibles*** = {MUR, SOL, MUR_PERMANENT, GHOT_HOUSE, TUNNEL, PORTE}
- Appel de `applyTemplate()` pour figer les structures fixes avant la génération
- Remplissage du reste de la grille avec des MUR prêts à être creusés.



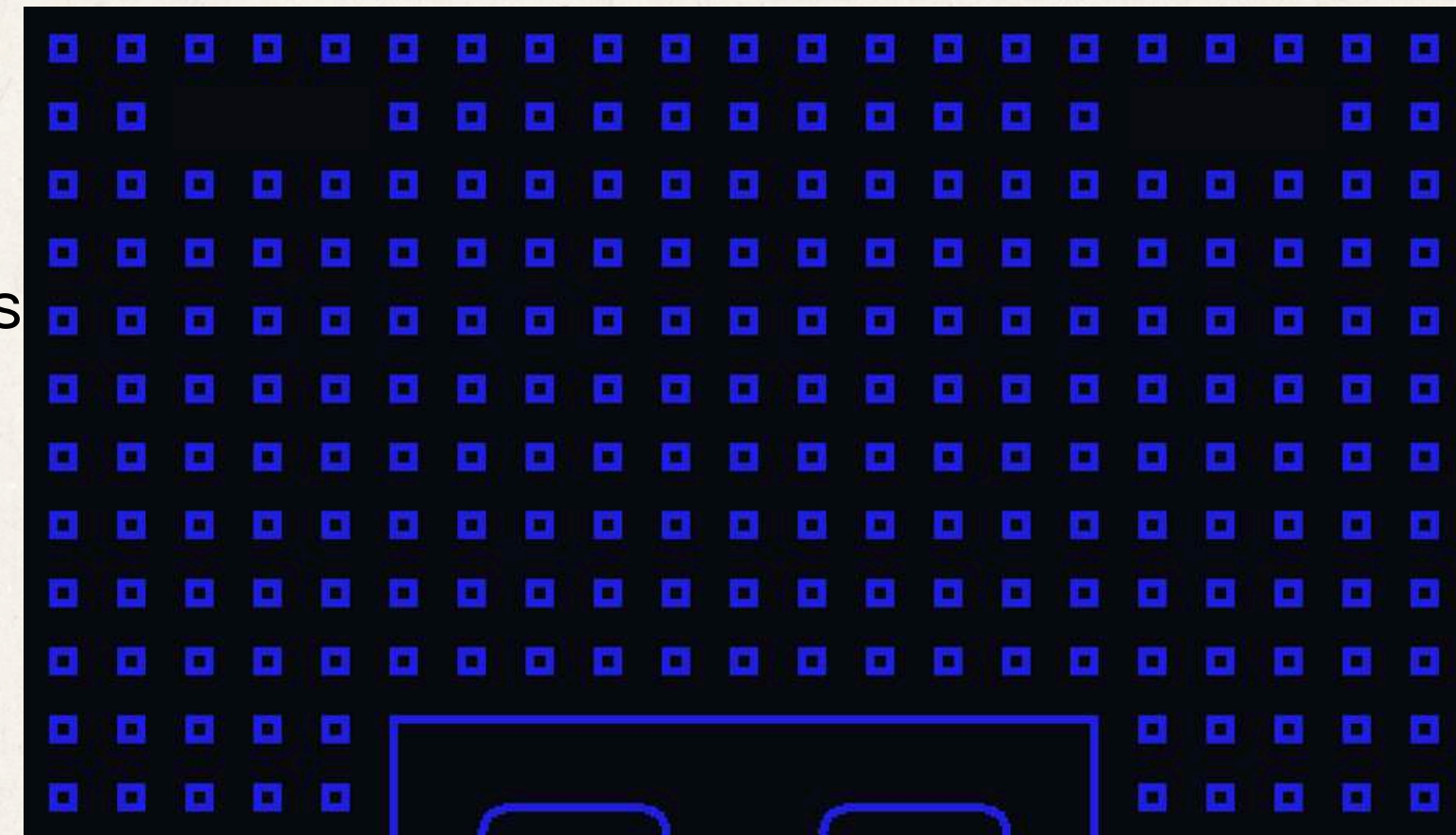
Etape 2 : Génération du labyrinthe

- **Structure WallPair** : Objet immuable (record) stockant un mur et son symétrique et les deux cellules que chaque mur sépare
- **Kruskal symétrique** : Algorithme de fusion d'ensembles (sets) utilisant WallPair pour garantir la symétrie



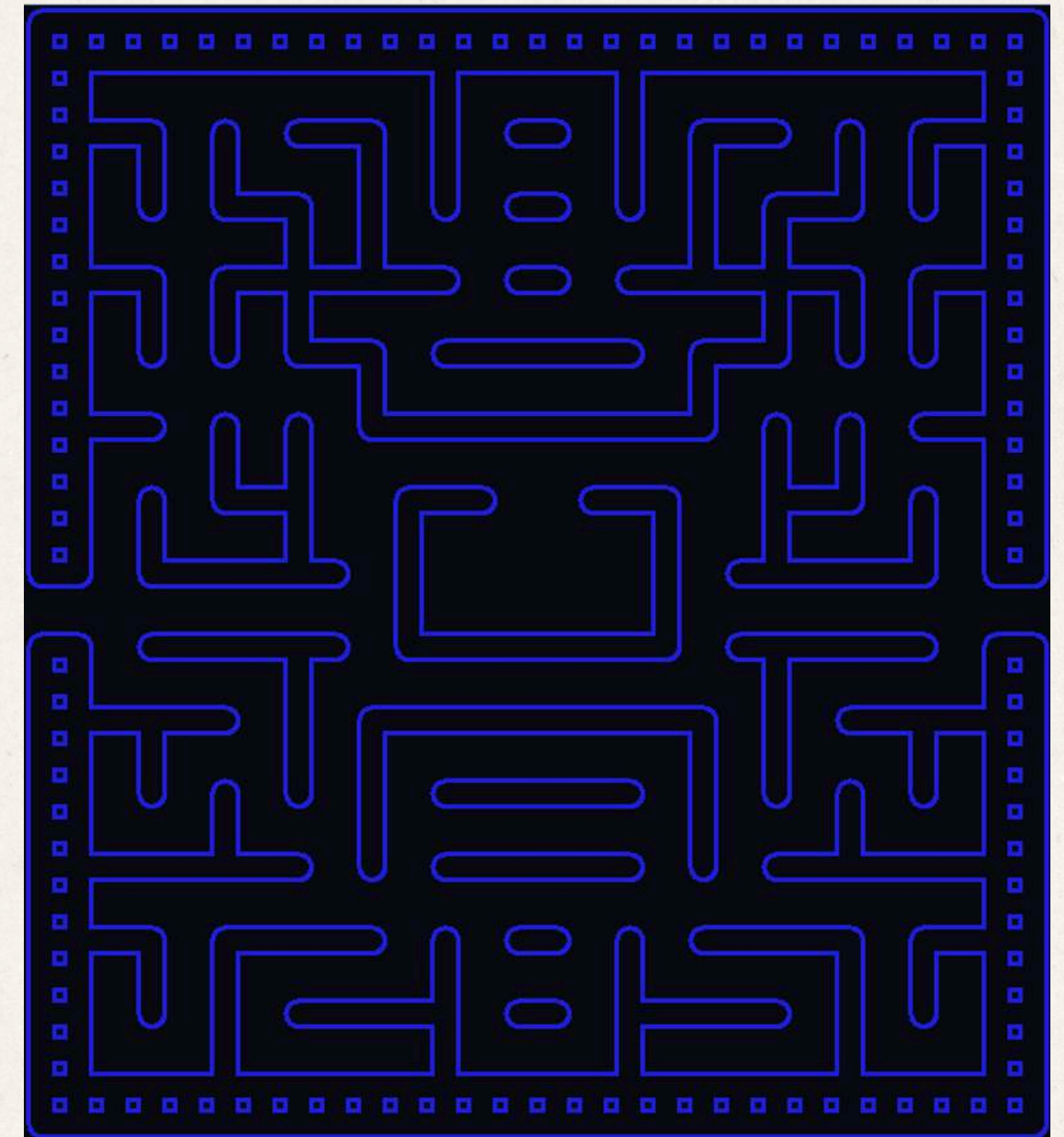
Etape 2 : Génération du labyrinthe

- **Structure WallPair** : Objet immuable (record) stockant un mur et son symétrique et les deux cellules que chaque mur sépare
- **Kruskal symétrique** : Algorithme de fusion d'ensembles (sets) utilisant WallPair pour garantir la symétrie
- **Méthode carvePassage** : Transforme simultanément 6 MUR en SOL pour créer un couloir symétrique



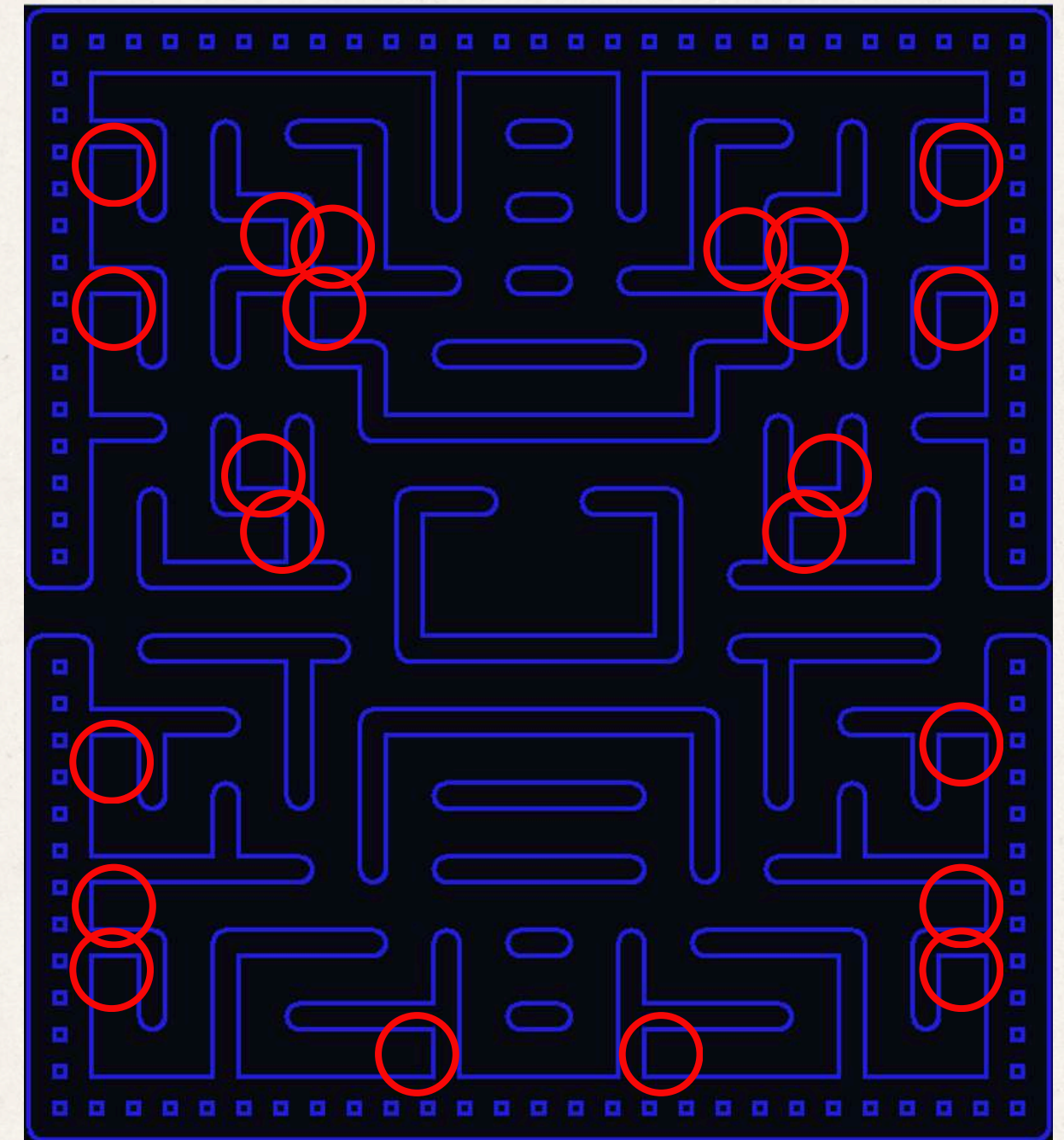
Etape 2 : Génération du labyrinthe

- **Structure WallPair** : Objet immuable (record) stockant un mur et son symétrique et les deux cellules que chaque mur sépare
- **Kruskal symétrique** : Algorithme de fusion d'ensembles (sets) utilisant WallPair pour garantir la symétrie
- **Méthode carvePassage** : Transforme simultanément 6 MUR en SOL pour créer un couloir symétrique
- **Graphe connexe** : Labyrinthe parfait donc...



Etape 2 : Génération du labyrinthe

- **Structure WallPair** : Objet immuable (record) stockant un mur et son symétrique et les deux cellules que chaque mur sépare
- **Kruskal symétrique** : Algorithme de fusion d'ensembles utilisant WallPair pour garantir la symétrie
- **Méthode carvePassage** : Transforme simultanément 6 MUR en SOL pour créer un couloir symétrique
- **Graphe connexe** : Labyrinthe parfait donc... rempli de culs-de-sac



Etape 3 : Suppression des culs-de-sac

- **Méthode *removeDeadEnds()*** : Identification des cellules SOL avec un seul SOL adjacent...



Etape 3 : Suppression des culs-de-sac

- **Méthode *removeDeadEnds()*** : Identification des cellules SOL avec un seul SOL adjacent... et destruction d'un MUR adjacent de manière aléatoire.

MUR_PERMANENT
donc ✗

MUR donc ✓



MUR donc ✓

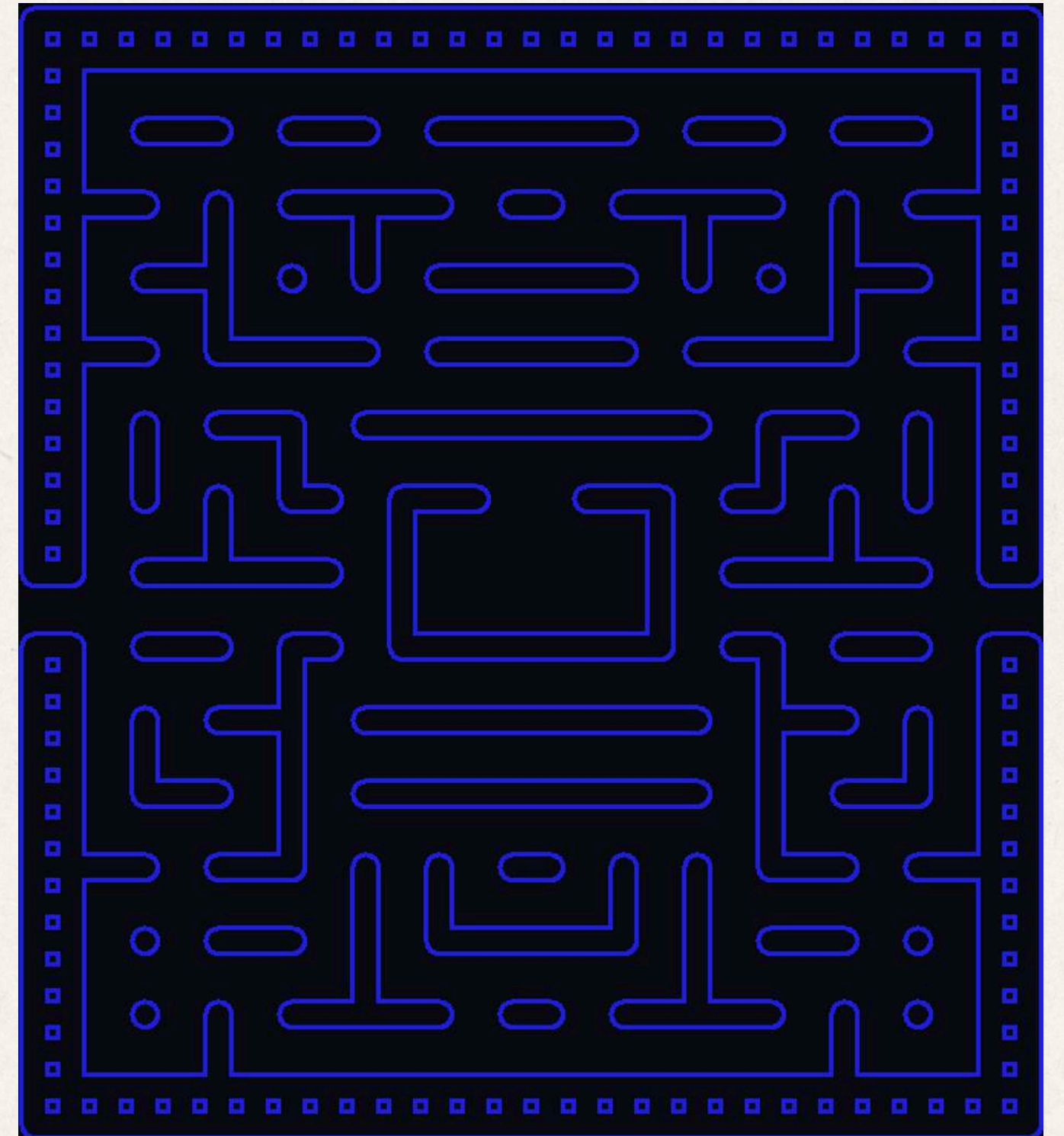
Etape 3 : Suppression des culs-de-sac

- **Méthode *removeDeadEnds()*** : Identification des cellules SOL avec un seul SOL adjacent... et destruction d'un MUR adjacent de manière aléatoire.



Etape 3 : Suppression des culs-de-sac

- **Méthode *removeDeadEnds()*** : Identification des cellules SOL avec un seul SOL adjacent... et destruction d'un MUR adjacent de manière aléatoire.



Infrastructure & DevOps :

Vue d'ensemble

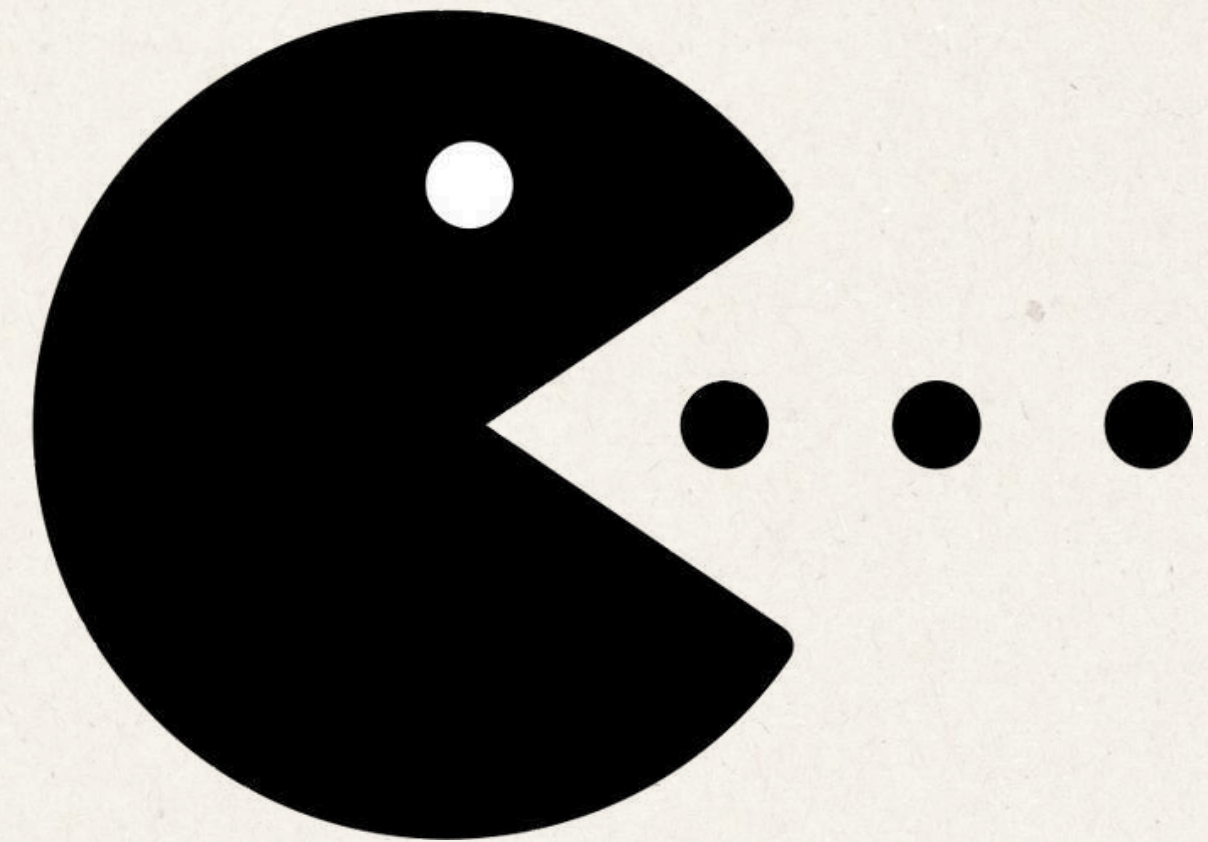
- **Architecture Client-Serveur** : Communication via API REST entre le client et le serveur Spark Java
- **Pipeline CI/CD** : Automatisation des tests unitaires et du build via Github Action pour garantir la stabilité du code
- **Conteneurisation & Cloud** : Déploiement du serveur via Docker sur la plateforme Render
- **Base de données(NoSQL)** : Utilisation de MongoDB Atlas pour le stockage des labyrinthes et des notations utilisateurs

Modélisation du Jeu : L'essentiel

- ***GameState (Le Modèle)*** : Centralise l'intégralité des variables à un instant T (labyrinthe, score, vies, positions des entités etc...)
- ***GameLogic (Le Moteur)*** : Centralise l'intégralité des règles physiques et des mécaniques du jeu (gestion des collisions, consommation des pastilles, mouvement des entités, mis à jour des positions etc...)

Vers un vrai jeu Pac-Man ?

- Algorithme glouton
 - Choix local
 - Choisir une distance
 - Pas de calcul du chemin total
- Targets du jeu original



Distance Euclidienne vs Manhattan

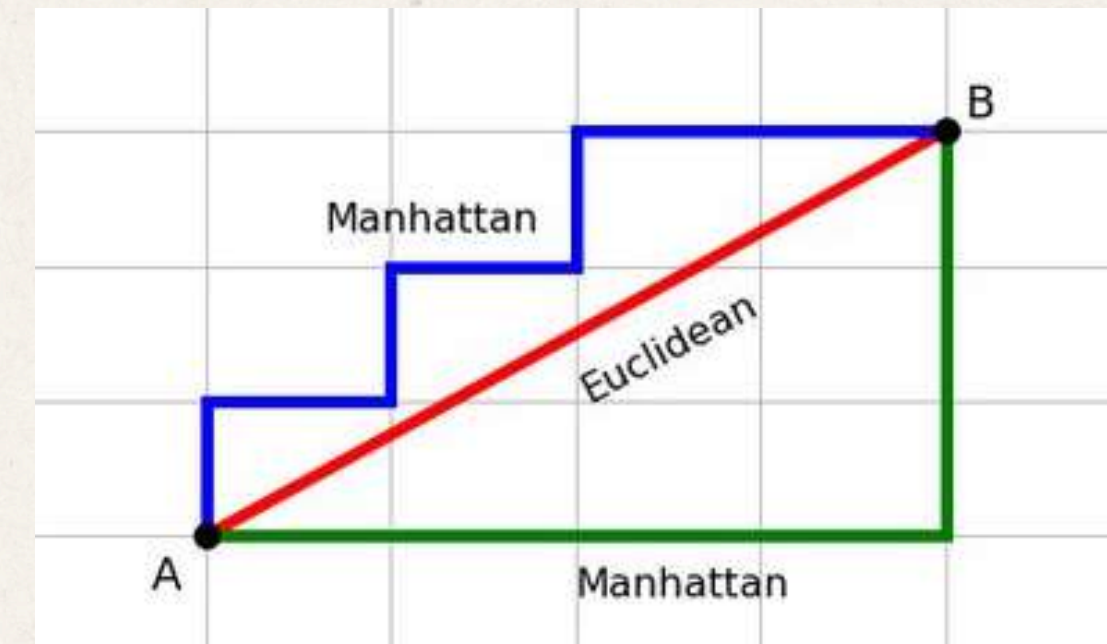
Manhattan :

- Déplacement orthogonaux
- Adaptée pour A*

Euclidienne :

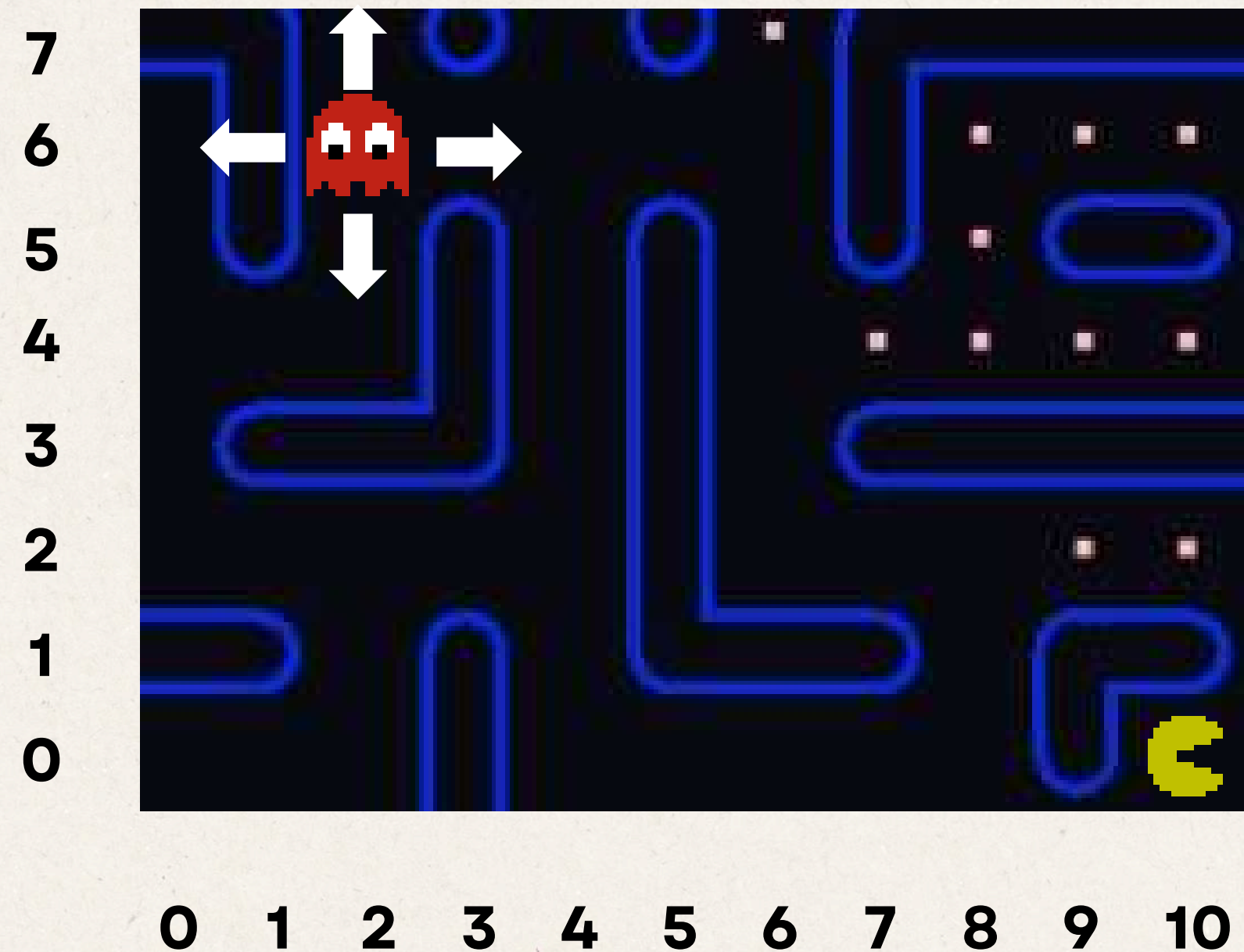
- Distance vol d'oiseau
- Pas de prise en compte des obstacles
- Plus cohérente avec algo glouton

$$d(A, B) = |X_B - X_A| + |Y_B - Y_A|$$



$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2}$$

Exemple d'exécution

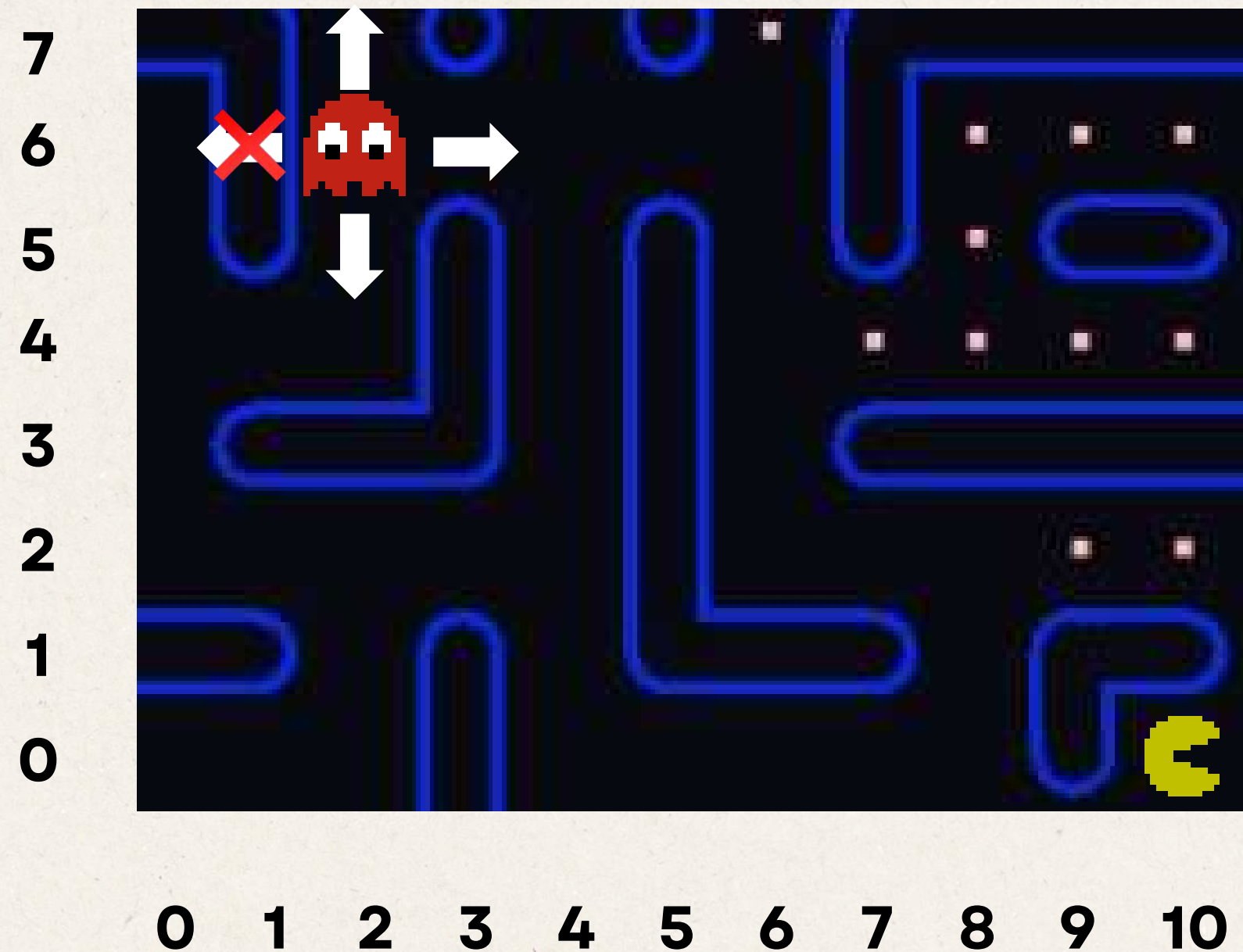


Fantome (2,6)

Pac-Man (10,0)

Vérification de l'état des cases
adjacentes

Exemple d'exécution



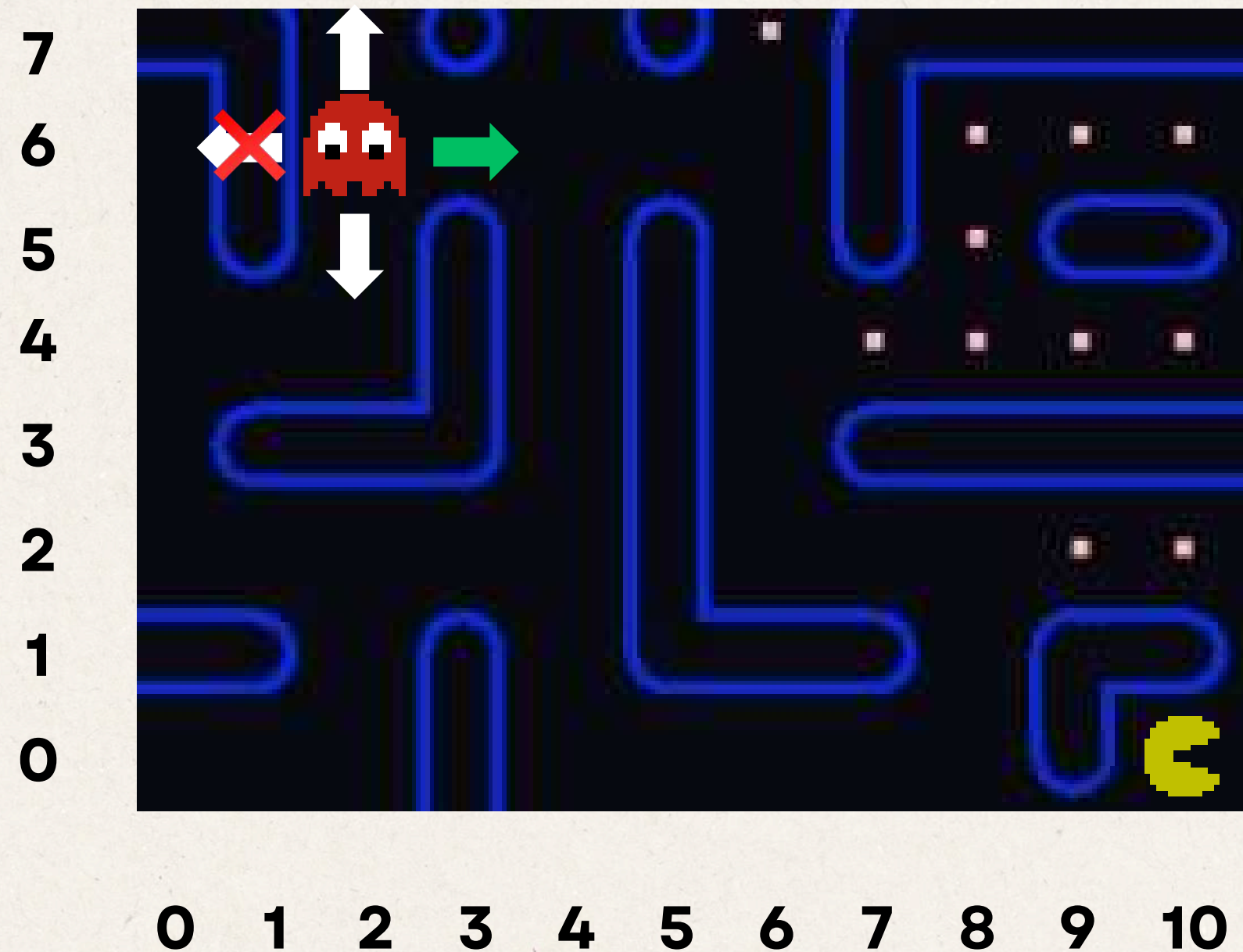
Fantome (2,6)

Pac-Man (10,0)

Calcul distance Euclidienne :

- Haut : (2,7)
 - $dx = 2 - 10 = -8$ / $dy = 7 - 0 = 7$
 - $\sqrt{(-8)^2 + 7^2} = \sqrt{64 + 49} = \sqrt{113} \approx 10.63$
- Droite : (3,6)
 - $dx = 3 - 10 = -7$ / $dy = 6 - 0 = 6$
 - $\sqrt{(-7)^2 + 6^2} = \sqrt{49 + 36} = \sqrt{85} \approx 9.22$
- Bas : (2,5)
 - $dx = 2 - 10 = -8$ / $dy = 5 - 0 = 5$
 - $\sqrt{(-8)^2 + 5^2} = \sqrt{64 + 25} = \sqrt{89} \approx 9.43$

Exemple d'exécution

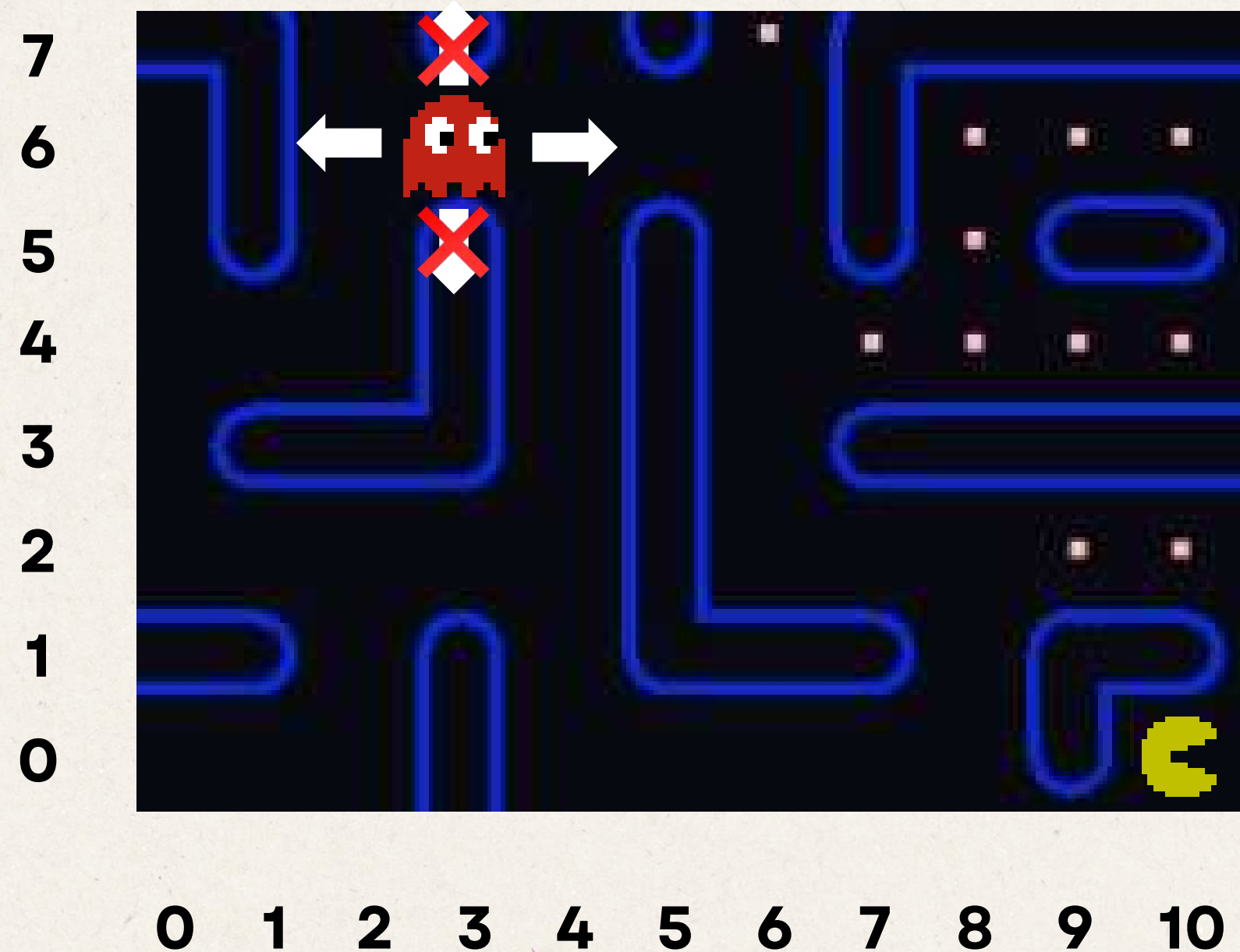


Fantome (2,6)

Pac-Man (10,0)

- Choix direction :
 - Droite (9.22) est le min donc on choisit la droite

Exemple d'exécution



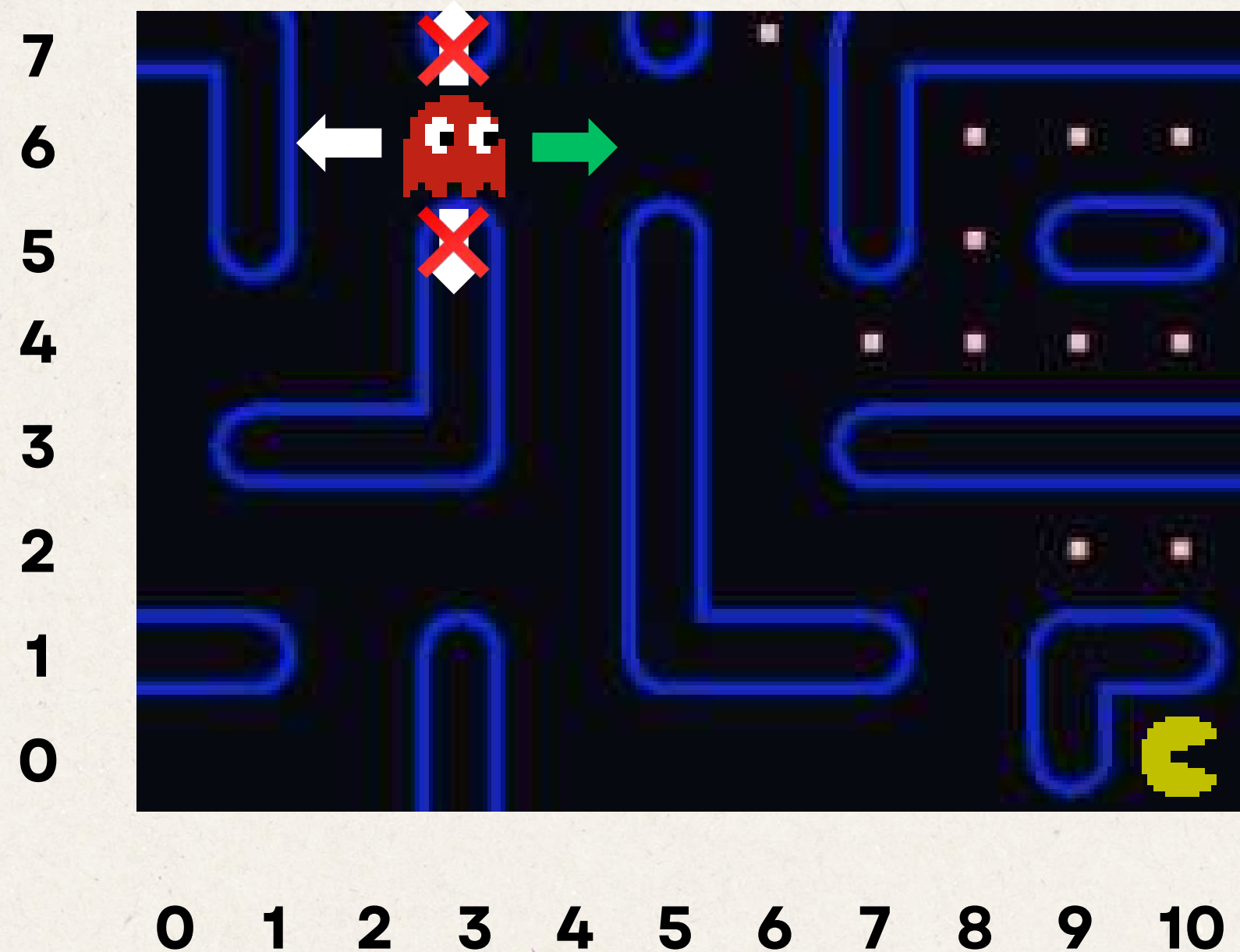
Fantome (3,6)

Pac-Man (10,0)

Calcul distance Euclidienne :

- Gauche : (2,6)
 - 10
- Droite : (4,6)
 - 8.49

Exemple d'exécution

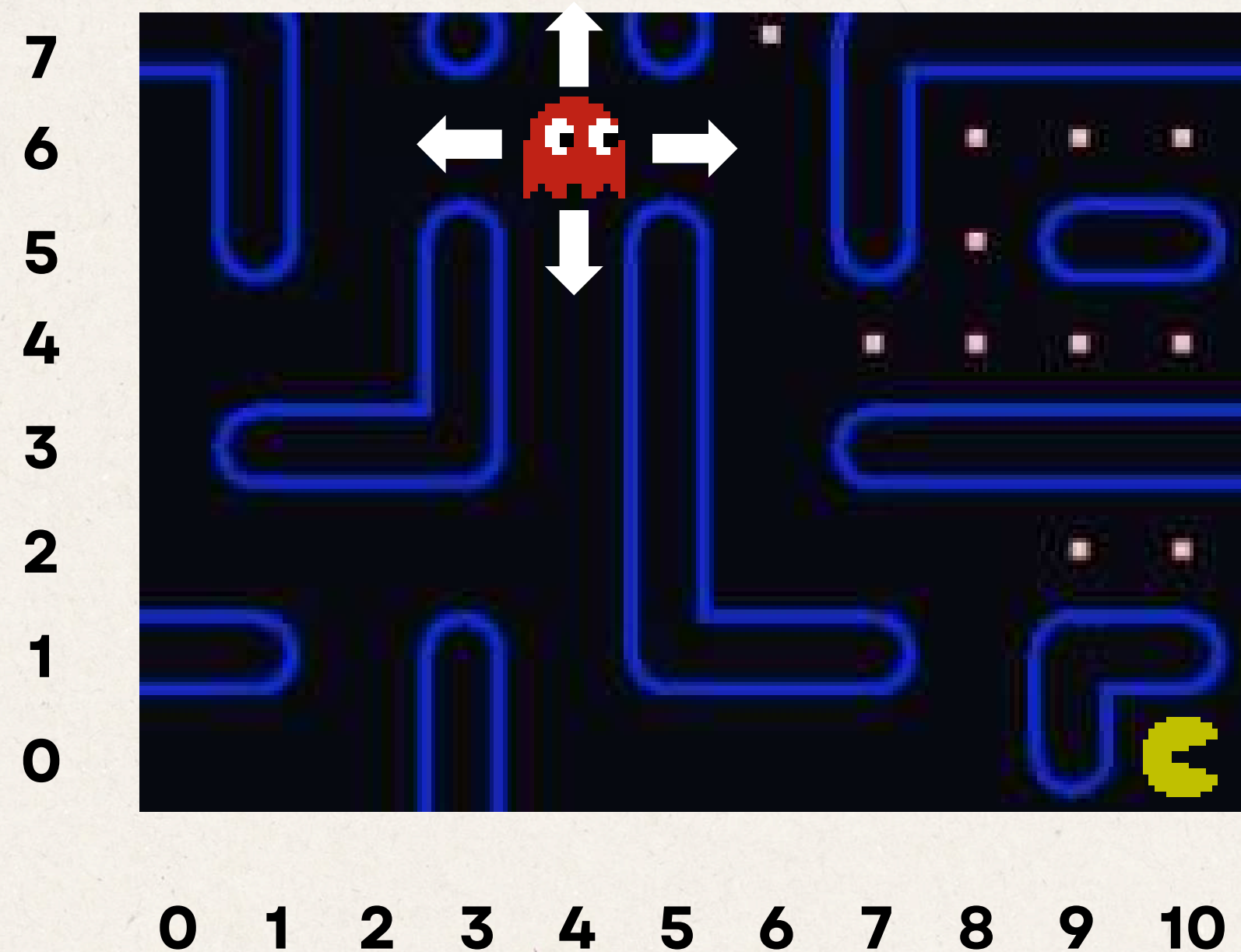


Fantome (3,6)

Pac-Man (10,0)

- Choix direction :
 - Droite (8.49) est le min donc on choisit la droite

Exemple d'exécution

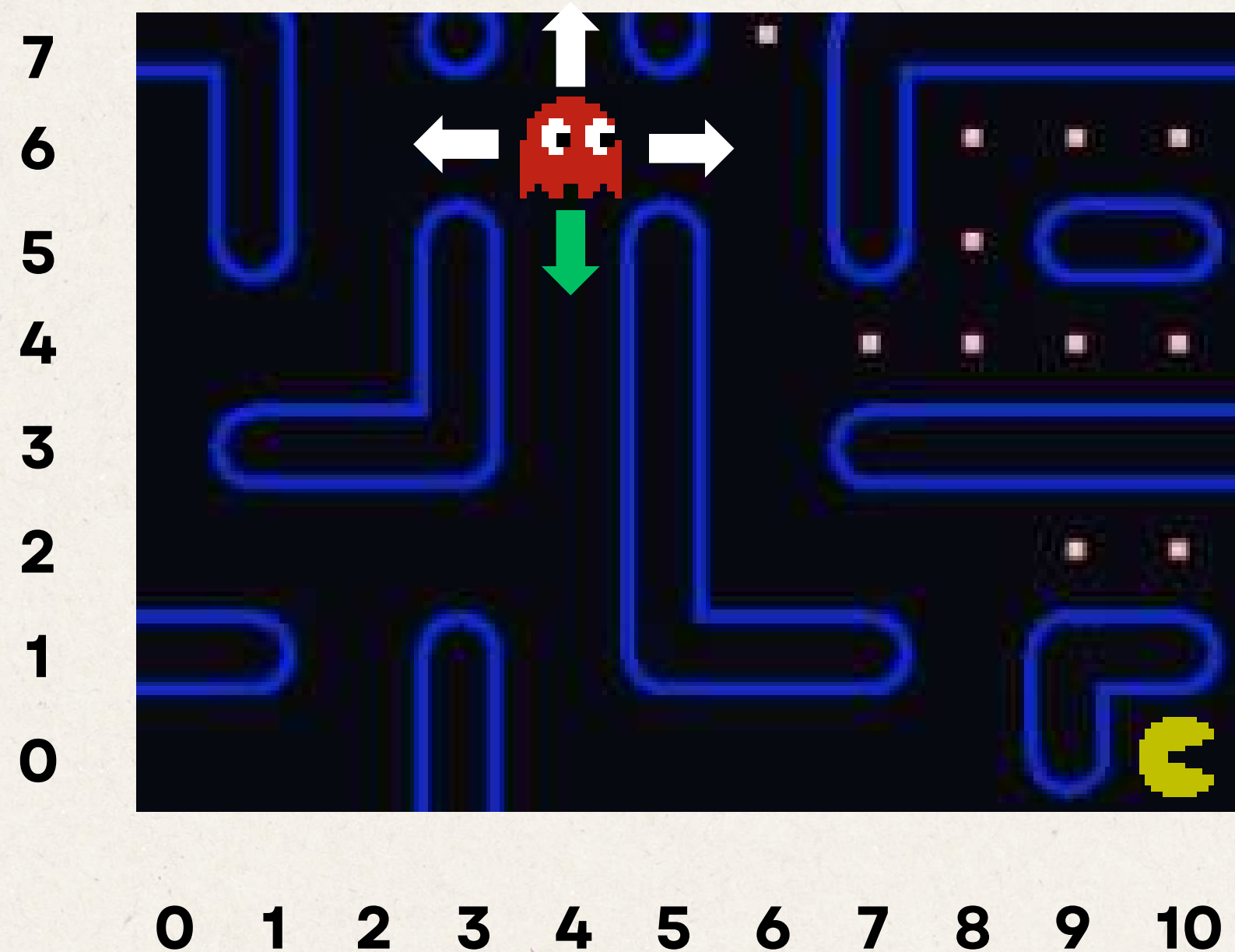


Fantome (4,6)
Pac-Man (10,0)

Calcul distance Euclidienne :

- Gauche : (3,6)
 - 9.22
- Bas (4,5):
 - 7.81
- Droite : (5,6)
 - 7.81
- Haut (4,7) :
 - 9.22

Exemple d'exécution



Fantome (4,6)

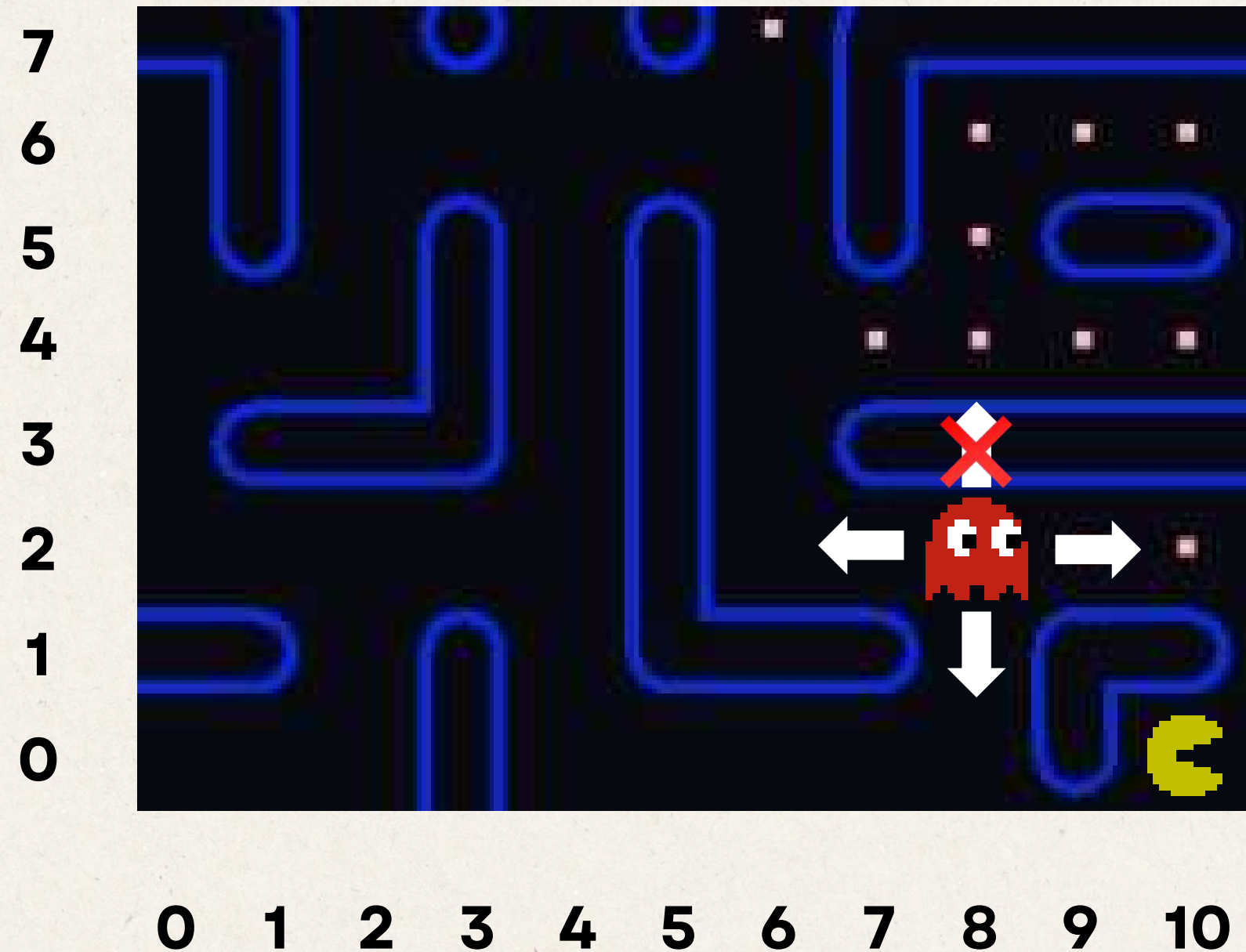
Pac-Man (10,0)

Choix direction:

- Droite et Bas (7.81) sont le min
 - Ordre comparaison algo :
 - Haut
 - Gauche
 - Bas
 - Droite

Donc on choisit le Bas

Limites de l'algorithme Glouton



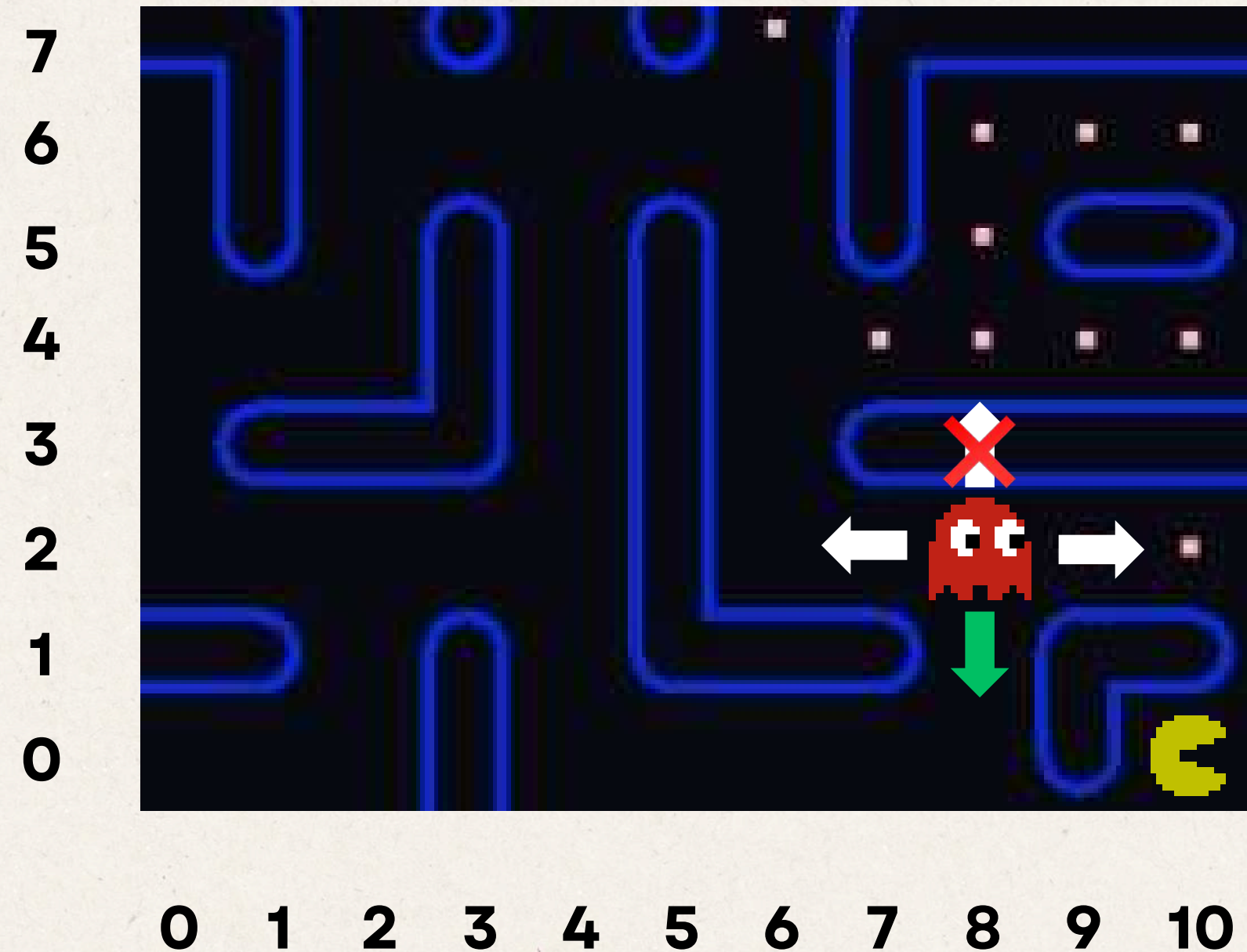
Fantome (8,2)

Pac-Man (10,0)

Calcul distance Euclidienne :

- Bas : (8,1)
 - 2.24
- Gauche : (7,2)
 - 3.61
- Droite : (9,2)
 - 2.24

Limites de l'algorithme Glouton



Fantome (8,2)

Pac-Man (10,0)

Calcul distance Euclidienne :

- Bas et Droite sont égaux (2.24)
 - Ordre comparaison algo :
 - Haut
 - Gauche
 - Bas
 - Droite
- On choisit donc Bas

Differentes targets

- Blinky (Rouge) : Vise Pac-Man directement
- Pinky (Rose) : +4 cases dans la direction de Pac-Man
- Inky (Bleu) : +2 devant Pac-Man en fonction de Blinky
- Clyde (Orange) : Vise Pac-Man et fuit si distance < 8

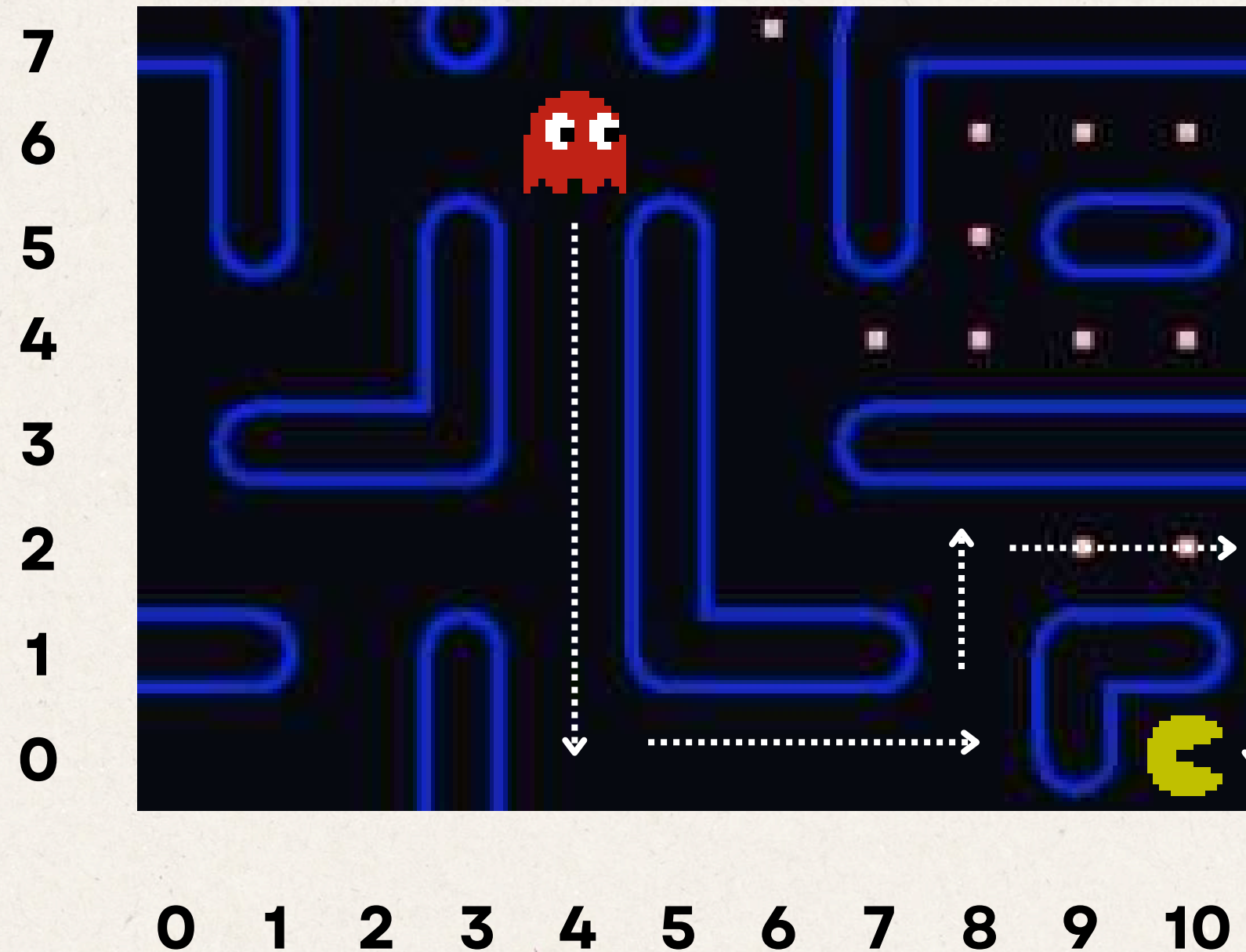
Avantages de cette méthode

- Se rapproche grandement du vrai jeu
- Pas besoin de calculer le chemin total
- Fantômes peu prévisibles

Principe de l'algorithme A*

- Calcul distance de manhattan
- Recherche du chemin optimal en tenant compte de l'ensemble du labyrinthe
- Évaluation de chaque case par un coût total :
distance parcourue + distance restante estimée

Exemple d'exécution



Fantome (4,6)

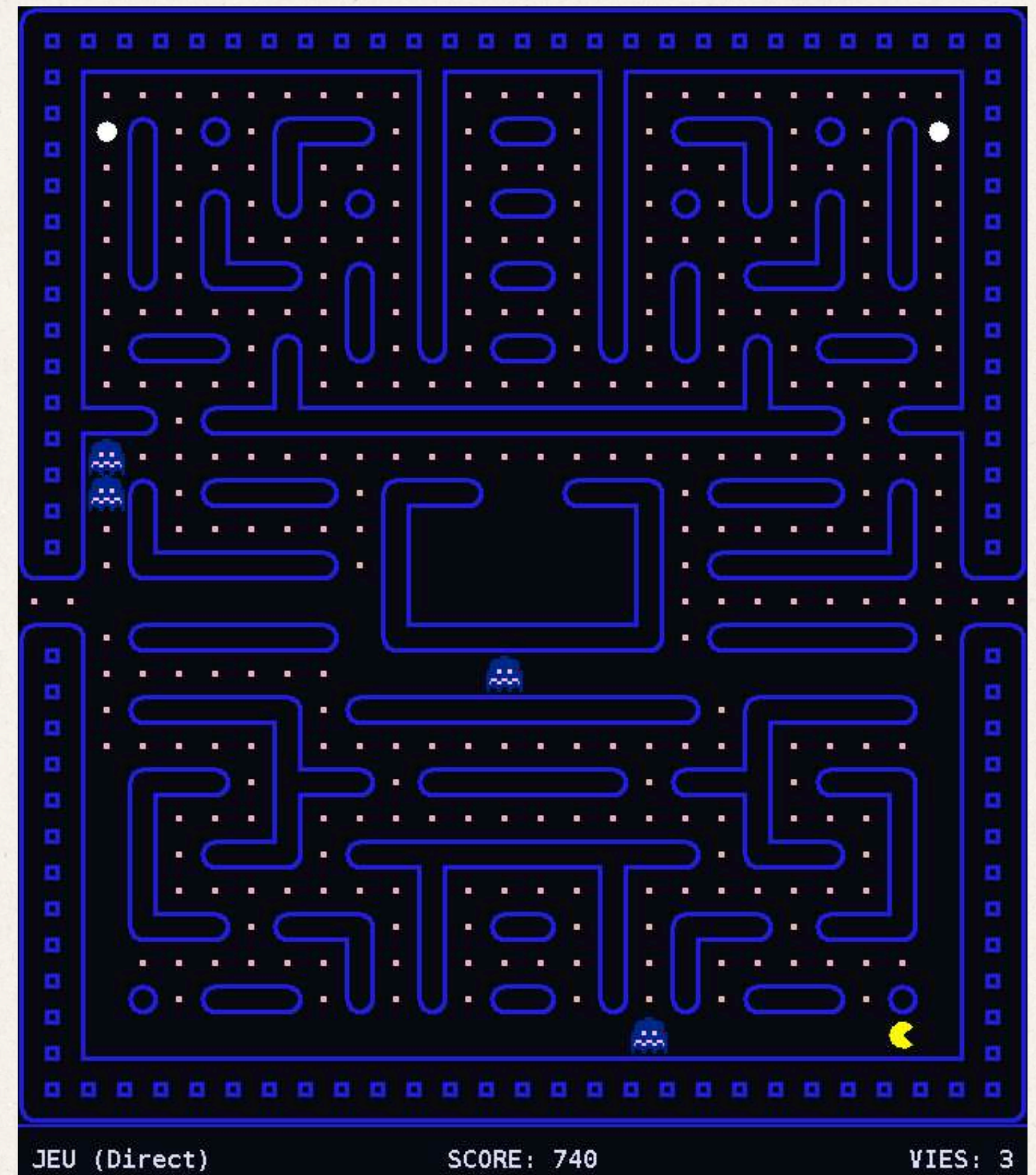
Pac-Man (10,0)

Calcul distance de manhattan :

- Haut (4,7) : $|4-10| + |7-0| = 6 + 7 = 13$
- Gauche (3,6) : $|3-10| + |6-0| = 7 + 6 = 13$
- Bas (4,5) : $|4-10| + |5-0| = 6 + 5 = 11$
- Droite (5,6) : $|5-10| + |6-0| = 5 + 6 = 11$

Fantômes éffrayés

- Déplacement Random
- Pas de demi-tour
- Retour à la GH si collision
- Clignotent quand la fin de l'effet approche

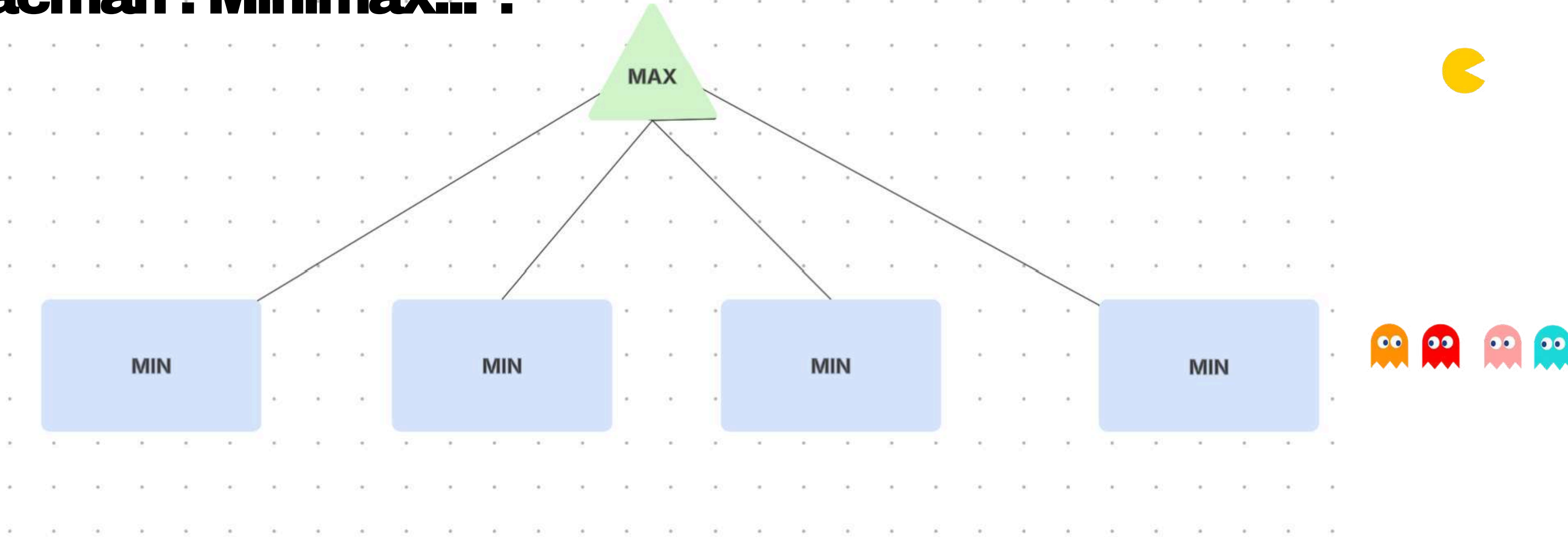


Fin de partie

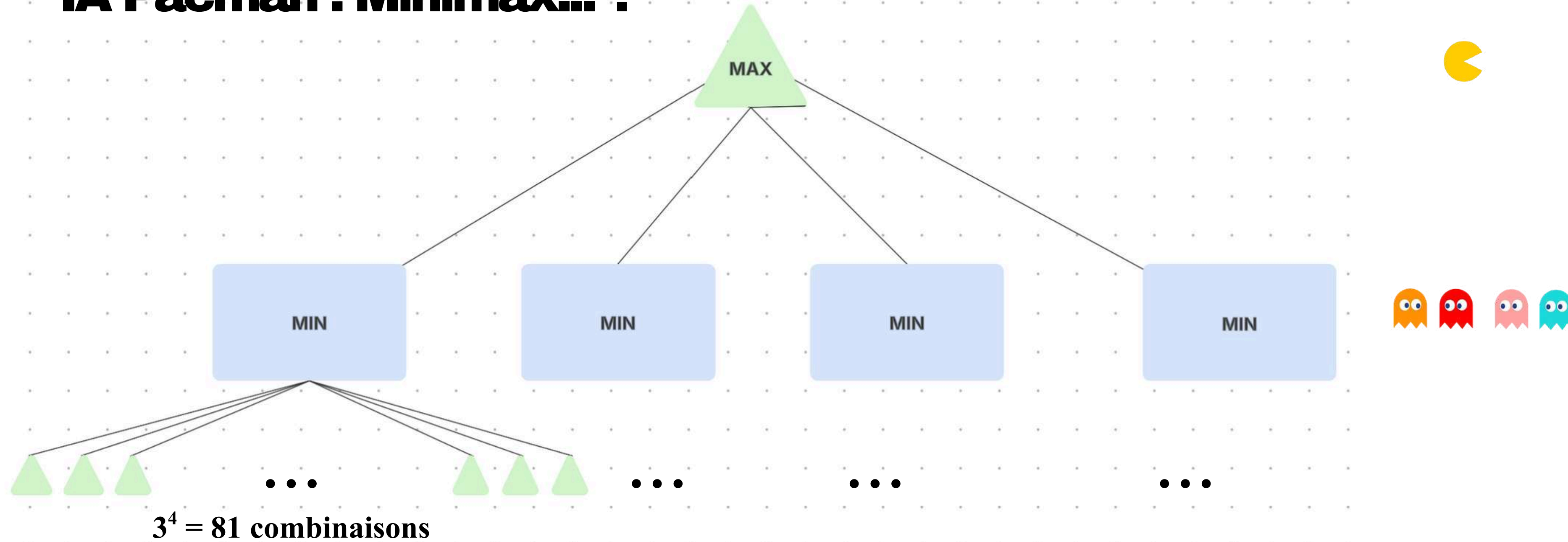
- 3 vies
- Si collision, retour à la ghost house, - 1 vie et pause
- Si croisement ?

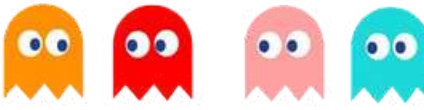


IA Pacman : Minimax... ?

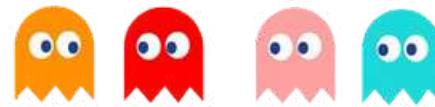
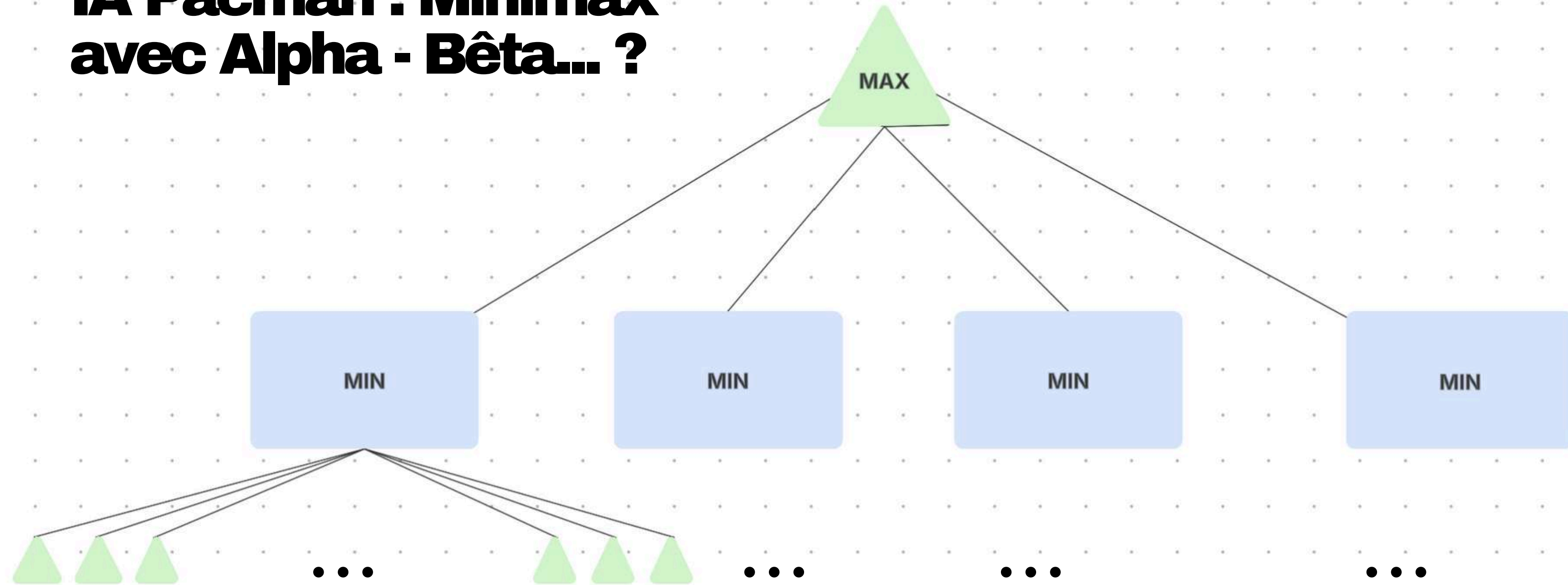


IA Pacman : Minimax... ?





IA Pacman : Minimax avec Alpha - Bêta... ?



$3^4 = 81$ combinaisons

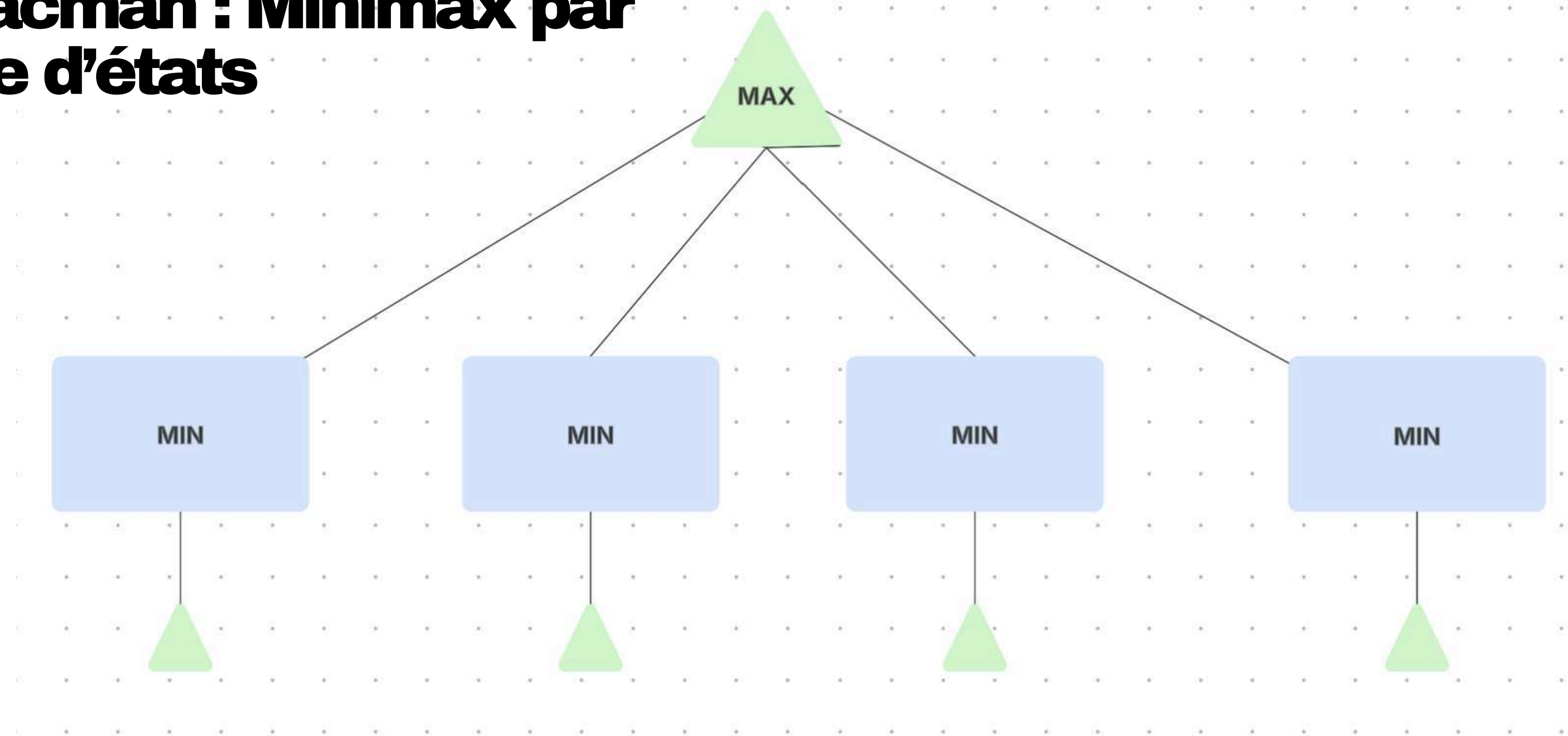
Nombre de visites optimal : $\sqrt{324} = 18$

Horizon 1

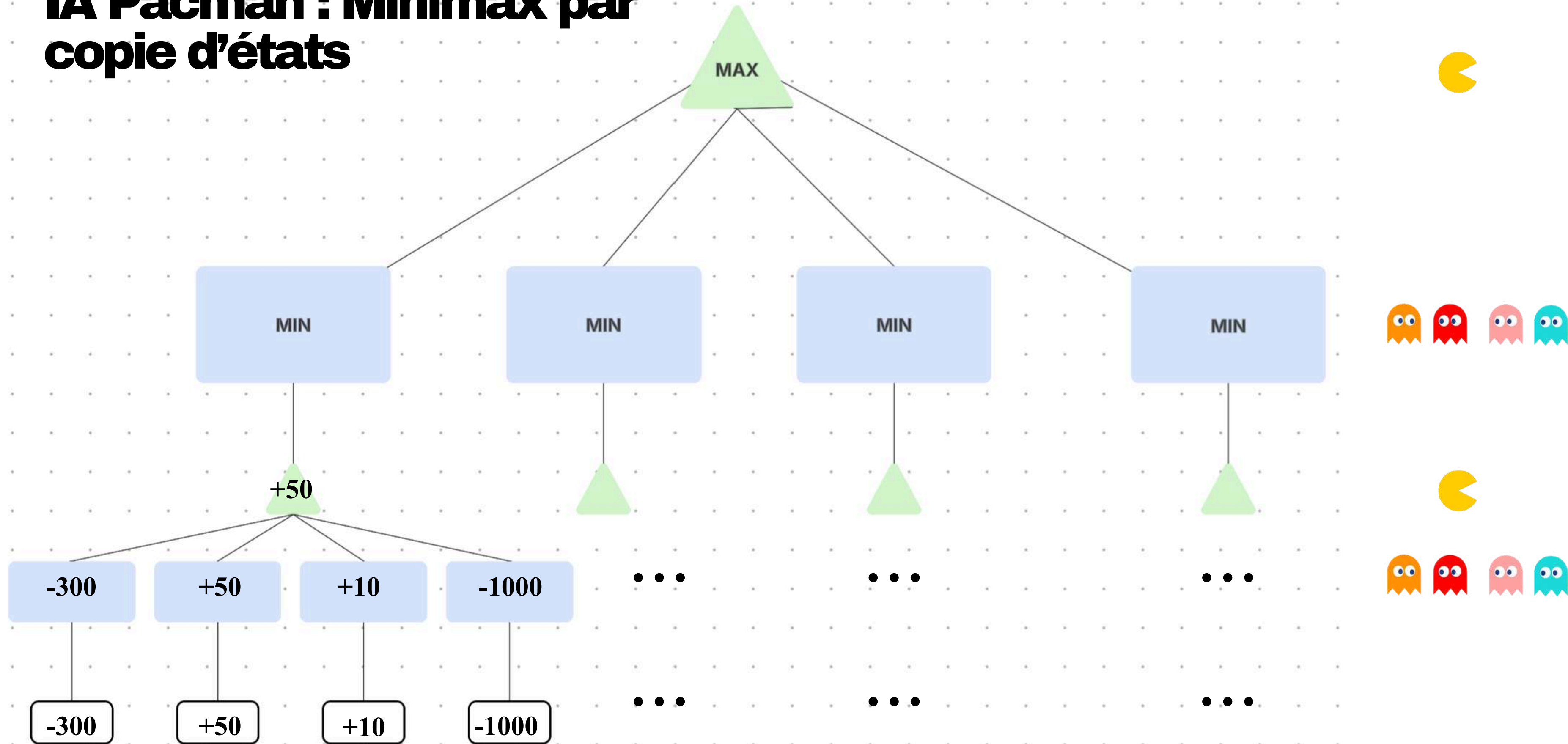
Nombre de visites optimal : $\sqrt{104\,976} = 324$

Horizon 2

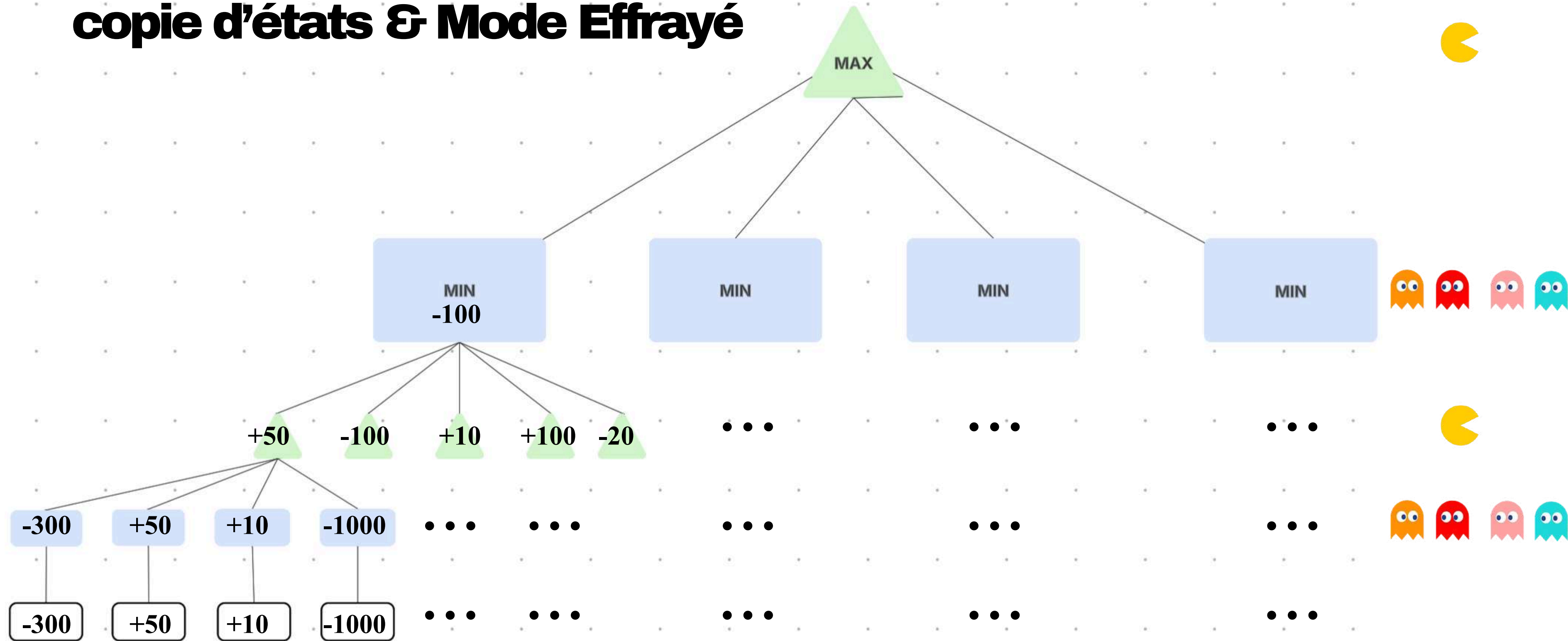
IA Pacman : Minimax par copie d'états



IA Pacman : Minimax par copie d'états



IA Pacman : Minimax par copie d'états & Mode Effrayé



L'impact de l'élagage alpha-bêta

Mode Normal

Horizon	Minimax (4^n)	Minimax avec Alpha-Beta (dans l'optimal $\sqrt{4^n}$)
1	4	2
2	16	4
3	64	8
4	256	16

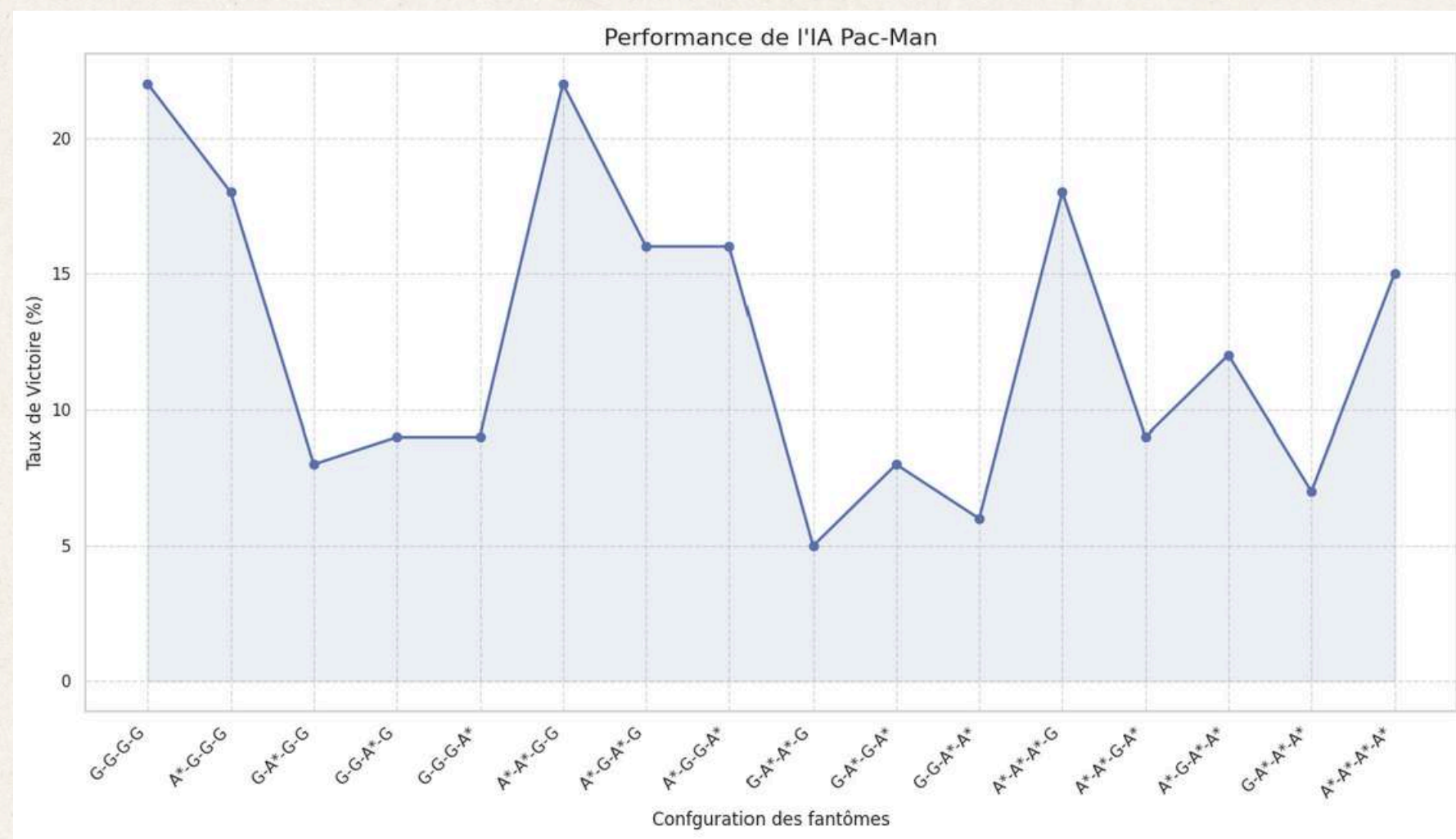
Mode Aléatoire (fantômes effrayés)

Horizon	Minimax (20^n)	Minimax avec Alpha-Beta (dans l'optimal $\sqrt{20^n}$)
1	20	~4-5
2	400	20
3	8000	~90
4	160 000	~400

Tests de Performances



- 16 configurations de fantômes à tester
- Exécution de 100 itérations par configuration :
 - IA vs IA : Evaluation du taux de succès de l'IA Pac-Man
 - IA vs Enregistrement : Evaluation du score moyen du Pac-Man



IA vs IA



IA vs Enregistrement